

IRE Assignment 2 Design Note

Learning from Click-Logs on EB-NeRD and MIND

Iman Kalyan Chakraborty - 2026801003

Shambhavi Gantla - 2026801014

September 17, 2026

<https://github.com/astrion-coder/cs4406m26-assignment1c1/>

1 Introduction

Assignment 1 built retrieval over a unified schema for EB-NeRD and MIND, using a from-scratch BM25 index and a frozen-embedding semantic retriever, but it learned nothing from the logs, because every score came from a fixed similarity formula. This assignment adds the behavioural half. Fifteen features are engineered from the click-logs (Section 2.1) and a LightGBM re-ranker is trained over them on top of A1’s retrieval scores (Section 2.2). Separately, the official **NRMSDocVec** baseline is reproduced from the authors’ own code and improved with one principled change, with a three-arm ablation and paired bootstrap intervals (Section 2.3), and the served system is measured rather than described (Section 2.4). Everything runs at both datasets’ large scale on one 15.7 GB desktop, with GPU-bound fitting offloaded to Kaggle. The re-ranker adds +0.0979 AUC over the best Stage-1 baseline on EB-NeRD and +0.0155 on MIND, and Section 3.2 argues that the gap between those two numbers, rather than their size, is the most informative thing measured here.

2 Design

2.1 Click-History and Session Features (Q1)

The feature set is fifteen columns, one row per (impression, candidate) pair. Five describe the user’s history, namely click count, recency-weighted category affinity, cosine similarity to a recency-weighted pool of history embeddings, and recency-weighted dwell time and scroll depth. Three describe the impression context, namely the candidate’s position in the in-view list and how many clicks and impressions the user has already had in this session. Three describe the article itself, namely add-one-smoothed training popularity, age at impression time, and whether its category appears in the user’s history. The remaining four are the Stage-1 retrieval signals discussed in Section 2.2.

Recency weighting is $w_i = \exp(-\ln 2 \cdot \Delta_i / H)$, but Δ_i means different things on the two datasets and the difference is recorded in the artifacts rather than hidden. On EB-NeRD it is real elapsed time in hours with a 72-hour half-life. On MIND it is rank from the most recent click with a five-click half-life, because MIND has no per-click timestamps anywhere in its raw data, and the metrics file marks this basis as an ordinal proxy so that no downstream reader can mistake it for elapsed time. The weights are recomputed per impression rather than cached per user, because they depend on that impression’s own timestamp, and the same history decays differently when

the user returns two days later. That is $O(\text{history length})$ work per impression and the dominant cost of the build, which takes 4h54m for 190,850,352 rows on `ebnerd_large`.

Five columns are null for every MIND row, being both dwell-time features, both session features, and freshness, because those signals do not exist in MIND’s logs at all. They are carried as nulls rather than zero-filled, since LightGBM routes a missing value at each split, so one feature set serves both datasets without a sentinel value the trees could read as a measurement, and Section 3.2 shows that this nullity is what separates the two datasets’ results. The behaviour-window boundary is enforced and then tested. A user’s history is a pre-collection-window snapshot by construction, the leakage test asserts that every consumed timestamp is strictly before the impression time, and a stronger check truncates one real user’s history to a prefix, recomputes three features by hand from that prefix alone, and requires the notebook’s own output to match. Deliberately excluded, and this is Q9’s substance, are the current impression’s own read time and scroll depth. They are strongly predictive, but they are the outcome of the impression being scored rather than a signal available before it.

2.2 The Re-Ranker (Q2)

Stage 2 is Option A, a LightGBM GBDT, and the decisive argument is structural. Q3 requires reproducing a neural baseline and beating it, so had Stage 2 also been a small neural ranker, the improvement under test and the thing it is measured against would have been the same class of model on the same signal, and a gain would have been uninterpretable. The feature set argues for trees independently, since fifteen heterogeneous columns on different scales, a third of them absent on one dataset, is exactly the shape where trees need no normalisation, imputation or embedding layer. Fitting is hosted on Kaggle and inference is local, the same split A1 used for embeddings and for the same reason, and what comes back is a 374 KB text file.

The training size was measured rather than asserted. The same configuration fitted on *nested* subsamples of 100k, 200k and 400k impressions, nested so that successive points differ only by added data, gives validation AUC of 0.6701, 0.6698 and 0.6655 on `ebnerd_large` and 0.6183, 0.6176 and 0.6158 on `mind_large`. The curve is flat to slightly declining on both, so 400,000 impressions is past the point where more data helps. Both models also stop at 53 and 45 trees against a 2,000-round budget, which is consistent with a feature set where a few columns carry nearly all the signal.

Why re-ranking operates on the in-view set

Q2 asks for A1’s candidate generator to retrieve top- K candidates which the re-ranker then re-orders, and this pipeline departs from that wiring. A1’s top-200 is scored against the whole catalogue of 125,541 EB-NeRD or 104,151 MIND articles, whereas an impression’s in-view set is what the publisher actually showed the user, about twelve articles on EB-NeRD and thirty-seven on MIND. The two sets barely intersect, since recall@200 against the clicked article is only 0.2 to 3%, so a second stage restricted to our top-200 would be re-ordering a list that usually does not contain the answer. The constraint is also external, because both leaderboards require the submission to be a permutation of the given in-view set, which makes a ranking of our own top-200 unsubmittable. The top-200 lists are therefore used as evidence instead, as two membership flags per candidate that overlap the in-view set 2.4 to 4.3 times above chance and carry exactly zero split gain in both trained models. This fork was identified during A1 (`SPEC.md`, A1 Q4, first section) rather than retrofitted here.

So Stage 1 genuinely feeds Stage 2, just through “how well does BM25 score this article for this

user” rather than “is this article in the user’s global top-200”. That is the same `score_inview` adapter A1 built, reused unchanged, which is why the two-stage framing still holds.

2.3 Baseline Reproduced, Then Beaten (Q3)

The baseline is `NRMSDocVec` from `ebanalyse/ebnerd-benchmark`, imported from a clone pinned at one commit and run unmodified, and the notebook asserts the checked-out revision and a clean working tree, so reproduction means the authors’ code rather than a reimplementaion of their paper. It runs on both datasets through the same code path, because A1’s unified schema and document vectors are exactly the input `NRMSDocVec` expects. Pairing it with `recommenders-team`’s NRMS-on-MIND instead would have meant maintaining two implementations and reporting MIND numbers not comparable to EB-NeRD’s.

The improvement is one term. `AttLayer2` pools a user’s history in a way that is blind to both order and time, and the variant adds a recency log-weight to the attention exponent, so that recency multiplies the learned attention rather than replacing it, using real elapsed time on EB-NeRD and the ordinal proxy on MIND. Those are A2 Q1’s own two bases rather than a third notion of recency. It adds no parameters, so all three variants have identical capacity, and it keeps the user vector candidate-independent, which is what makes the scoring shortcut in Section 2.5 possible.

The ablation has three arms rather than two, because the recency layer also masks the zero-padded history slots the baseline attends over, and that is a real fix rather than the recency signal. The middle arm, `masked_uniform`, is the same architecture fed all-zero log-weights, so the two paired intervals separate the mask from the signal, and the three point estimates decompose exactly, which the notebook asserts. All three variants carry an identical 1,304,720 parameters, so nothing below is capacity.

dataset	split	baseline	masked_uniform	recency
<code>ebnerd_large</code>	val	0.5701	0.5773	0.5757
<code>ebnerd_large</code>	test	0.5835	0.5836	0.5845
<code>mind_large</code>	val	0.6316	0.6323	0.6396
<code>mind_large</code>	test	0.6307	0.6261	0.6349

Table 1: Per-impression AUC, 200,000 impressions per split, identical impressions across variants.

On MIND the improvement is confirmed, since all four metrics on both splits give a paired 95% interval that excludes zero and is positive, with Δ AUC of +0.0080 [+0.0071, +0.0090] on val and +0.0042 [+0.0033, +0.0051] on test. On EB-NeRD it is not, because val agrees at +0.0056 [+0.0046, +0.0066] but test does not, with AUC and nDCG@5 indistinguishable from zero and MRR and nDCG@10 significantly negative. It is reported as such, so the assignment’s bar is met for MIND on both splits and for EB-NeRD on val only.

What the middle variant buys is the reason why. Isolating recency from the padding mask it also introduces, the two datasets attribute the same headline change to opposite causes. On MIND the masking does nothing on val and hurts on test, so the ordinal recency prior is the whole effect, worth +0.0073 and +0.0088. On EB-NeRD the val gain *is* the mask, worth +0.0072, and the elapsed-time recency prior removes 0.0016 of it. A two-arm experiment would have reported EB-NeRD’s val result as a confirmed recency improvement and been wrong about the mechanism.

Against Q2’s numbers, measured on the same 200,000 impressions per split and therefore directly comparable, the two datasets invert. On `mind_large` the improved NRMS reaches test AUC

0.6349 and beats the two-stage re-ranker at 0.6099, while on `ebnerd_large` it is far behind at 0.5845 against 0.6490. A neural encoder over title embeddings is the stronger model exactly where the behavioural log is thin, which is the same finding Section 3.2 reaches from the other direction.

One alternative explanation for EB-NeRD’s val and test disagreement should be stated rather than left implicit. The authors’ early-stopping rule monitors the last calendar day of the training split, and EB-NeRD’s 07:00 cutoff leaves that rule only 3.8% of the training sample to monitor on, against MIND’s 25.4%. A stopping signal computed on a twenty-fifth of the data is a plausible cause of a variant that confirms on val and not on test, and it is not something the ablation design can separate. The supporting figures are `nrms_ablation.png` and `nrms_paired.ci.png`.

2.4 Serving and Scale Analysis (Q4)

The served path consists of BM25 and embedding scoring over one impression’s in-view set, feature assembly, and LightGBM inference, and it was timed over 1,000 sampled requests after 50 warm-up requests on this machine (i5-13500, 15.7 GB). Against the brief’s 100 ms example SLA, p99 latency is **6.09 ms** on `ebnerd_large` and **10.43 ms** on `mind_large`, which is 16 and 9.6 times of headroom, 230 and 161 queries per second per process, and \$0.000051 and \$0.000073 per thousand queries at an assumed \$0.0425 per vCPU-hour. The resident serving set is 0.80 and 0.68 GB, of which the embedding matrix and its normalised copy are some 90%, while the BM25 index is 84 and 85 MB and the booster 374 and 320 KB.

Three findings matter more than the headline. First, `bm25_inview` is 83% and 88% of the served path, because an inverted index scores the whole catalogue to read out a dozen documents, so the index choice that made A1’s candidate generation fast is the single thing holding the served path above a millisecond, and a forward index would remove it. Second, the offline feature table already exceeds this machine at 1× on `ebnerd_large`, at 27.9 GB against 15.7 GB, and at 2× on MIND. That is the measured form of every memory incident in the project and the reason Q1 and Q2 stream it. It is an offline artifact, since a live system derives features from history and article metadata, so the timed feature assembly is a lower bound. Third, calling A1’s offline `batched_top_k` once per request re-normalises the corpus on every call, making it 40 times slower than the serving-shaped matrix-vector product at 311 and 274 ms p99, which alone would breach the SLA at 1×. That was measured deliberately, as the price of reusing an offline function for serving.

2.5 Code Optimizations

The performance work that mattered was memory rather than speed, and one bug class recurred six times, namely converting an Arrow column to Python objects before reducing it. Each instance was fixed by staying columnar, for instance by taking `list.to_array(dim).to_numpy()` instead of a comprehension over `to_list()`, which was worth 3.5 GB of transient memory on the blind EB-NeRD build alone. The others are listed below.

- **Peak stacking.** Releasing the previous dataset’s index before building the next, rather than assigning over the binding, which keeps both alive through the allocation. BM25 vocabulary order uses `sorted(set(...))` so that builds stay reproducible.
- **Streaming joins.** Two eager `collect` calls against a 164M-row exploded top-200 frame killed the kernel, whereas `sink_parquet` over the same plan bounds the peak regardless of how large the other side is.

- **Join direction.** Polars builds its hash table on the right side of a left join, so joining a 200k-row slice against a 13.5M-row table hashed the wrong side, and a semi-join first cuts it to the slice’s rows in 0.3s.
- **External sort.** A plain sort of the 13.5M-row blind behaviors peaked at 10.1 GB, whereas a range-bucketed external sort reproduces it bit-for-bit at 2.55 GB.

Q3 contributes one more. The `ebnerd-benchmark` scoring path flattens every (impression, candidate) pair into its own row, re-encoding the user’s whole twenty-click history once per candidate. Since `NRMSDocVec` scores by a dot product of a history-only user vector with an article-only news vector, the news encoder can run once over the catalogue and the user encoder once per impression, removing a factor of roughly 11.9 on EB-NeRD and 37 on MIND of encoder work. The notebook validates the shortcut against `model.scorer.predict` directly, checking that the sigmoid of the dot product agrees within 1e-4 with zero ranking flips, rather than assuming the algebra. This is also why the Q3 improvement was designed to keep the user vector candidate-independent, since a candidate-aware encoder would hand the factor back, offline and at serving time alike.

3 Discussion

3.1 Results

All three methods were scored on one common sample of 200,000 impressions per split, which is what makes the differences paired. The sample is not assumed to be representative. The two Stage-1 baselines were re-measured on it and agree with A1’s full-population values to within 0.0008 on every ranking metric, and with A1’s ILD and novelty inside the sample’s own 95% interval on every split.

dataset	method	AUC	MRR	nDCG@5	nDCG@10
<code>ebnerd_large</code>	<code>bm25</code>	0.5015	0.3191	0.3495	0.4344
	<code>embedding</code>	0.5511	0.3486	0.3867	0.4648
	<code>reranker</code>	0.6490	0.4224	0.4722	0.5334
<code>mind_large</code>	<code>bm25</code>	0.5569	0.3000	0.2760	0.3378
	<code>embedding</code>	0.5944	0.3196	0.2975	0.3613
	<code>reranker</code>	0.6099	0.3341	0.3140	0.3747

Table 2: Before and after re-ranking, test split, 200,000 impressions, identical impressions across methods.

Every paired interval excludes zero. Against the stronger baseline the re-ranker gains +0.0979 AUC [+0.0964, +0.0996] on `ebnerd_large` test and +0.0155 [+0.0145, +0.0166] on `mind_large` test, and against BM25 it gains +0.1475 and +0.0530. The val split agrees in both direction and magnitude, at +0.1066 and +0.0057 against embedding, so the effect is not a property of one week of logs.

3.2 Why the Gain Differs Between the Two Datasets

The same model, the same fifteen features and the same hyperparameters produce +0.0979 AUC on EB-NeRD and +0.0155 on MIND, and the explanation lies in the logs rather than in the modelling.

Seven of the fifteen features carry zero split gain on `mind.large` against two on `ebnerd.large`, and the single most informative EB-NeRD feature, `freshness_hours` at 48% of total gain, is one of the five columns MIND never recorded. MIND’s booster instead leans on `popularity` at 53% and `position_in_impression` at 21%, both of which are weak and user-independent signals. The ceiling on what a re-ranker can learn is set by what the log records rather than by the learner, and on this evidence instrumenting dwell time, sessions and publication timestamps would buy more than any change of model.

3.3 Extended Evaluation and Slices

Beyond accuracy the picture is mixed and worth reporting as such. On the test split the re-ranker has the lowest intra-list diversity of the three methods on both datasets, at 0.781 against 0.798 and 0.791 on EB-NeRD and 0.785 against 0.854 and 0.838 on MIND, and its top-10 catalogue coverage is equal or lower. Novelty, however, moves in opposite directions, falling on MIND to 20.10 against 20.38 and 20.62 but *rising* on EB-NeRD to 22.33 against 21.96 and 22.14. The mechanism is the feature importances again, because novelty is $-\log_2$ of training popularity, fresh articles have no training clicks, and EB-NeRD’s ranker is freshness-driven while MIND’s is popularity-driven. The claim that the re-ranker recommends less novel items would be false on one of the two datasets.

Slices expose one genuine failure. On a temporal split the head is small, at 1.5% of EB-NeRD test impressions, since head membership is defined by training clicks and news decays, and on that slice the re-ranker is **below chance**, with AUC 0.416 [0.404, 0.427] against BM25’s 0.596. The mechanism was measured rather than guessed. Clicked head articles are a median 239 hours old against 3.1 hours for tail, the freshest in-view candidate is typically an hour old, and the re-ranker ranks the clicked article a median 7th of 11, so a freshness prior buries the evergreen story the user came back for. On val, one day after training, the same model leads on head, which separates being wrong about head articles from being wrong about *old* articles. Cold-start exists only on MIND, where both retrieval baselines score exactly 0.5000 because an empty query is a full tie, and the re-ranker reaches 0.511 on popularity and position alone.

3.4 Where This Breaks at 10x

The order in which this system breaks at ten times the load is measured rather than guessed, and the first component has already broken. The offline feature table reaches 279 and 101 GB at 10 $\times$, against 27.9 GB today on a 15.7 GB machine, so it is streamed and chunked precisely because it never fit, and at 10 $\times$ it would need a columnar store queried per request rather than a materialised table. Brute-force embedding retrieval goes second, because the serving set reaches 8.0 and 6.8 GB and corpus-wide top-200 p99 reaches 81 and 69 ms, so an approximate index stops being an optimisation and becomes a requirement. BM25 corpus-wide top- K goes third and actually breaches the SLA on MIND, at 146 ms p99 with 37 postings per document against EB-NeRD’s 22, which WAND or sharding would address. The re-ranker never breaks, since it scales with the in-view set, which is a property of the publisher’s page rather than of the corpus, and it is the cheapest stage in the system at every scale measured. The supporting figure is `serving_benchmark.png`.

4 Leaderboard Submissions

Both blind populations, being 13,536,710 EB-NeRD and 2,370,727 MIND impressions and 205.9 M and 93.1 M candidate rows, were scored with the full two-stage pipeline. Having no labels, they take their popularity basis from the training dataset’s lookup re-keyed into their own article namespace.

The feature table is never joined to the Stage-1 scores, because both notebooks emit rows in one deterministic order of user and impression and align 200,000-impression chunks by position, which also lets the BM25 adapter’s one-entry cache score each user once rather than once per impression, at 4,356 impressions per second against roughly 430 for Q2’s non-contiguous sample.

8	lmankc	2026-09-16 18:27	927493	927493_RecSys24_competition_v5	0.6547	0.428	0.4802	0.5404	
63	lmankc	2026-09-14 22:00	926252		0.6223	0.3016	0.3234	0.3793	

Both were scored. The RecSys 2024 Challenge returns AUC 0.6547 with MRR 0.428, nDCG@5 0.4802 and nDCG@10 0.5404, placing 8th of 269 entries, and MIND returns AUC 0.6223 with MRR 0.3016, nDCG@5 0.3234 and nDCG@10 0.3793. These corroborate the offline evaluation rather than restate it, being measured on weeks the pipeline never saw, with 0.6547 against the 0.6490 held-out AUC of Section 3.1 and 0.6223 against 0.6099. The EB-NeRD entry is also directly comparable with A1’s, which scored the same blind population at AUC 0.5141 using BM25 retrieval, so the two-stage pipeline adds **+0.1406 AUC** on the leaderboard itself. That is a before-and-after measured by the organisers rather than by this project’s own harness, and it is consistent in direction with the +0.0979 held-out gain.

One operational note belongs here, since it cost more time than any modelling decision. Codabench dispatches jobs to a participant-run compute worker, and the current server encodes the submission secret in kombu 5.x’s wrapper format while naming bundles per phase, so every published worker image disagrees with one or both conventions. This worker logged 26 failed status updates out of 26 attempts across 14 jobs and three weeks, each failing before the submission file was downloaded. Three jobs did reach the running state, which is why the affected submissions sat as submitted rather than erroring outright. Since the queue is shared between participants, A1’s one scored EB-NeRD entry was handled by another participant’s worker. Three one-line compatibility shims produced the first successful run in this worker’s history.